

Introdução à C/C++ para Programadores Python

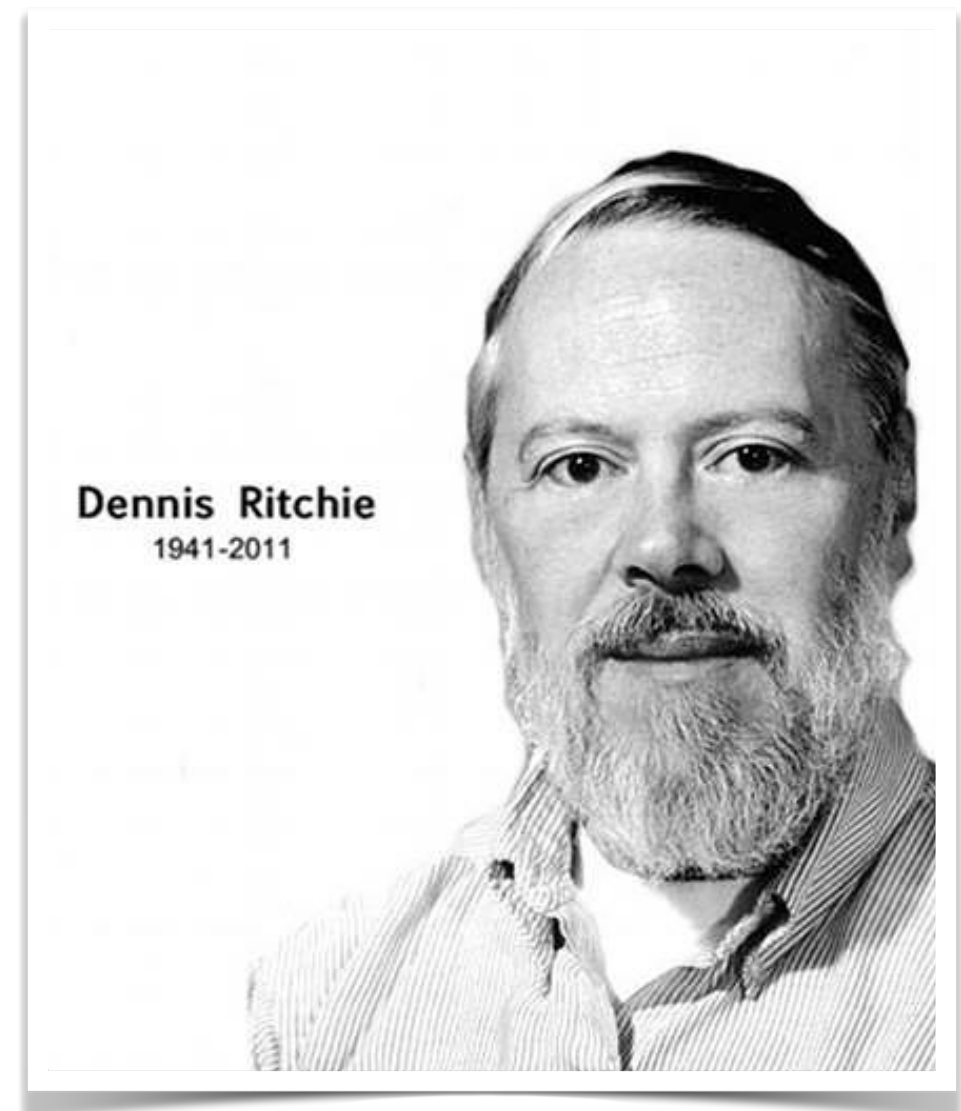
Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

História C/C++

- A **linguagem de programação C** foi criada no início da década de 1970 por Dennis Ritchie no Bell Labs
 - Objetivo principal de desenvolver o sistema operacional UNIX
- **C** é baseada na linguagem B, de Thompson, evolução da antiga linguagem BCPL
- **C++** é uma evolução de **C** (~80s)



Abstração

Programa em
linguagem de
alto nível
(em C)

```
swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compilador

Programa em
assembly
(para MIPS)

```
swap:
    muli $2, $5, 4
    add  $2, $4, $2
    lw   $15, 0($2)
    lw   $16, 4($2)
    sw   $16, 0($2)
    sw   $15, 4($2)
    jr   $31
```

Montador

Programa
binário em
linguagem de
máquina
(para MIPS)

```
00000000101000010000000000011000
0000000000001100000001100000100001
10001100011000100000000000000000
100011001111001000000000000000100
10101100111100100000000000000000
101011000110001000000000000000100
00000011111000000000000000001000
```

Primeiro Programa

```
1. // Um primeiro programa em C/C++
2. #include <stdio.h>
3.
4. int main() {
5.
6.     printf("Ola mundo!\n");
7.
8.     return 0;
9. }
```

```
% g++ hello_world.cpp
% ./a.out
Ola mundo!
%
```

Códigos Especiais	Significado
<code>\n</code>	Nova linha.
<code>\t</code>	Tabulação.
<code>\\</code>	\ - Barra invertida.
<code>\0</code>	Zero.
<code>\'</code>	Aspas simples (apóstrofo).
<code>\"</code>	Aspas dupla.

Principais códigos especiais em C/C++.

Outro Programa: Adicione Dois Inteiros

```
1. // Programa de adicao
2. #include <stdio.h>
3.
4. int main() {
5.     int a, b, soma; // declare
6.
7.     printf("Entre com um inteiro\n"); // imprima
8.     scanf("%d", &a); // leia um inteiro
9.
10.    printf("Entre com outro inteiro\n"); // imprima
11.    scanf("%d", &b); // leia um inteiro
12.
13.    soma = a + b; // atribuicao de soma
14.    printf("A soma eh %d\n", soma); // imprima soma
15.
16.    return 0; // programa finaliza com sucesso
17.}
```

```
Entre com um inteiro
50
Entre com outro inteiro
5
A soma eh 55
```

Códigos de formatação para print()	Significado
%c	Caractere simples.
%d	Inteiro decimal com sinal.
%f	Ponto flutuante em decimal.
%lf	Ponto flutuante de dupla precisão.
%s	String de

Exemplo 1: Múltiplas Leituras c/ Scanf()

```
1. #include <stdio.h>
2.
3. int main () {
4.     float p1, p2, p3, p4;
5.     double media;
6.
7.     printf("Digite as notas das 4 provas: ");
8.     scanf("%f%f%f%f", &p1, &p2, &p3, &p4);
9.     media = (p1 + p2 + p3 + p4)/4;
10.    printf("MEDIA: %.2f\n", media);
11.
12.    return 0;
13.}
14.
```

Códigos de formatação para scanf()	Significado
%c	Caractere simples.
%d	Inteiro decimal com sinal.
%f	Ponto flutuante em decimal.
%lf	Ponto flutuante de dupla precisão.
%s	String de caracteres.

```
Digite as notas das 4 provas: 5 6 7 8
MEDIA: 6.50
```

Códigos Formatação Printf

Códigos de formatação para printf()	Significado
%c	Caractere simples.
%d	Inteiro decimal com sinal.
%i	Inteiro decimal com sinal.
%e	Notação científica (e minúsculo).
%E	Notação científica (E maiúsculo).
%f	Ponto flutuante em decimal.
%g	Usa %e ou %f, o que for menor.
%G	Usa %E ou %f, o que for menor.
%o	Inteiro octal sem sinal.
%s	String de caracteres.
%u	Inteiro decimal sem sinal.
%x	Inteiro hexadecimal sem sinal (letras minúsculas).
%X	Inteiro hexadecimal sem sinal (letras maiúsculas).
%p	Ponteiro (endereço).
%n	Ponteiro inteiro.
%%	Imprime o caractere %.

Códigos Formatação Scanf

Códigos de formatação para scanf()	Significado
%c	Caractere simples.
%d	Inteiro decimal com sinal.
%i	Inteiro decimal, hexadecimal ou octal.
%e	Notação científica.
%f	Ponto flutuante em decimal.
%g	Usa %e ou %f, o que for menor.
%o	Inteiro octal.
%s	String de caracteres.
%u	Inteiro decimal sem sinal.
%x	Inteiro hexadecimal.
%ld	Inteiro decimal longo.
%lf	Ponto flutuante longo (double).
%LF	Double longo.

Operador de Endereços (&)

- Toda variável ocupa uma certa localização na memória
 - Seu **endereço** é o do primeiro *byte* ocupado por ela
- Um **endereço** é a referência que o computador usa para localizar as variáveis
- O operador **&** opera sobre o nome de uma variável e resulta o seu **endereço**

Exemplo 2: Endereço da Variável

```
1. #include <stdio.h>
2.
3. int main () {
4.     int n;
5.
6.     n = 2;
7.     printf("Valor=%d, endereco=%p\n", n, &n);
8.
9.     return 0;
10. }
```

Valor=2, endereco=0x16f0373e8

Variáveis

- Aspecto fundamental de qualquer linguagem
- É um espaço de memória reservado para armazenar um certo **tipo** de dado e tendo um **nome** para referenciar o seu conteúdo
- Toda variável em C deve ser declarada antes de ser usada!!

Tipos de Variáveis

Tipo	Bits	Bytes	Escala
char	8	1	128 a 127
int	32	4	-2.147.483.648 a 2.147.483.647 (ambientes de 32 bits)
float	32	4	$3,4 \times 10^{-38}$ a $3,4 \times 10^{38}$
double	64	8	$1,7 \times 10^{-308}$ a $1,7 \times 10^{308}$
void	0	0	nenhum valor

Tipos de variáveis em C.

- C++ adiciona o tipo **bool** (*true*, *false*) aos tipos primitivos de C

Operadores Aritméticos

- C oferece os seguintes operadores aritméticos:
 - **Binários:** soma (+), subtração (-), multiplicação (*), divisão (/) e módulo (%)
 - **Unário:** menos unário (-)

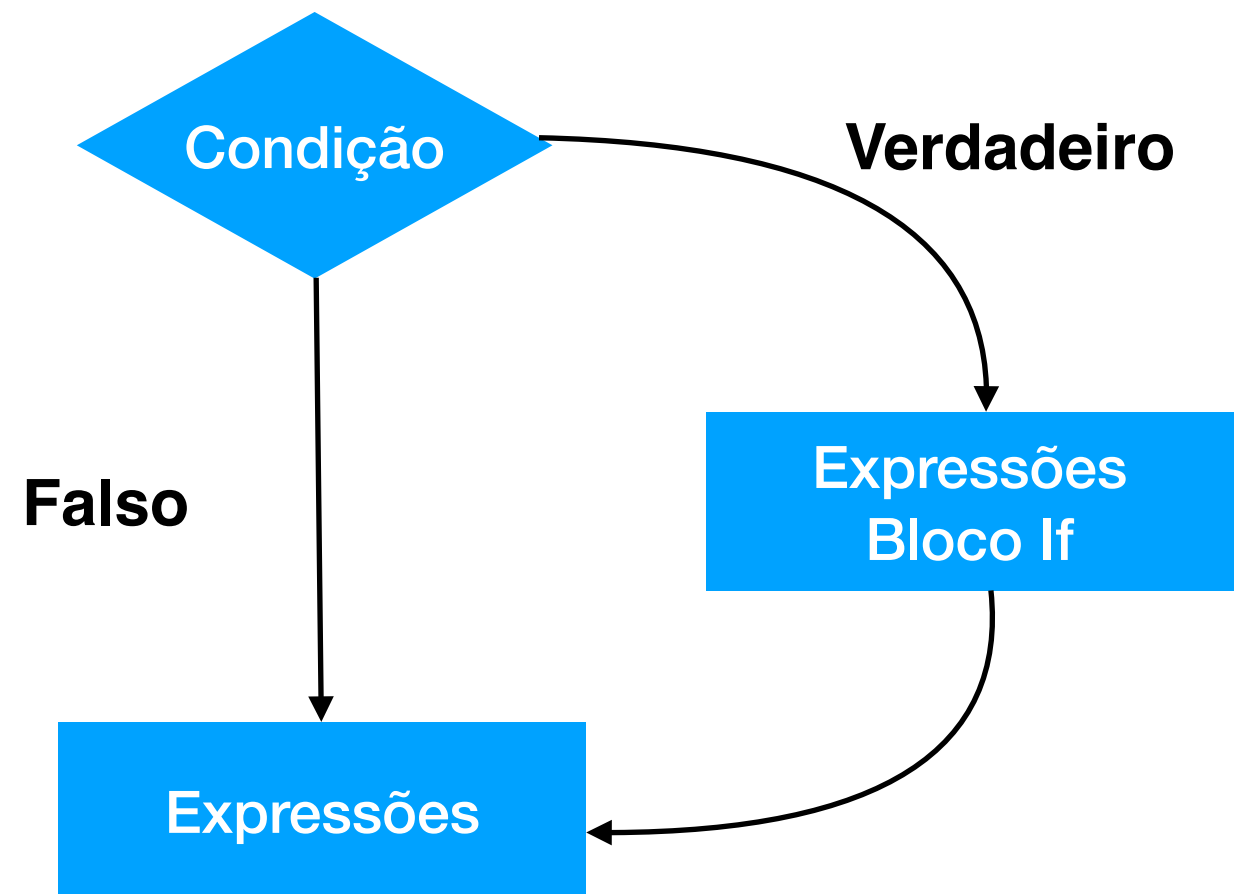
Operador	Exemplo	Comentário
<code>+=</code>	<code>x += y</code>	Equivale a <code>x = x + y</code> .
<code>-=</code>	<code>x -= y</code>	Equivale a <code>x = x - y</code> .
<code>*=</code>	<code>x *= y</code>	Equivale a <code>x = x * y</code> .
<code>/=</code>	<code>x /= y</code>	Equivale a <code>x = x / y</code> .
<code>%=</code>	<code>x %= y</code>	Equivale a <code>x = x % y</code> .

É possível combinar operador aritmético com operador de atribuição.

Comandos de Decisão

Se (*if*): Desvio Condicional Simples

```
int main() {  
    if (<condição>) {  
        ...  
    }  
    return 0;  
}
```



Exemplo 3: Aprovado?

```
1. #include <stdio.h>
2.
3. int main() {
4.     float media_final;
5.
6.     printf("Qual a sua media final da disciplina?\n");
7.     scanf("%f", &media_final);
8.
9.
10.    // Toma uma decisão se nota maior ou igual a 6.0
11.    if (media_final >= 6.0) {
12.        printf("Aprovado!\n");
13.    }
14.
15.    return 0;
16.}
```

```
Qual a sua media final da disciplina?
7
Aprovado!
Qual a sua media final da disciplina?
5
```

Operadores Relacionais

- A **condição** em estruturas **if** podem ser formadas utilizando os operadores de igualdade e operadores relacionais
- O resultado obtido de uma relação é sempre um valor lógico (**0** para falso ou **1** para verdadeiro)

Operador	Exemplo	Comentário
==	x == y	O conteúdo de x é igual ao conteúdo de y.
!=	x != y	O conteúdo de x é diferente do conteúdo de y.
<=	x <= y	O conteúdo de x é menor ou igual ao conteúdo de y.
>=	x >= y	O conteúdo de x é maior ou igual ao conteúdo de y.
<	x < y	O conteúdo de x é menor que o conteúdo de y.
>	x > y	O conteúdo de x é maior que o conteúdo de y.

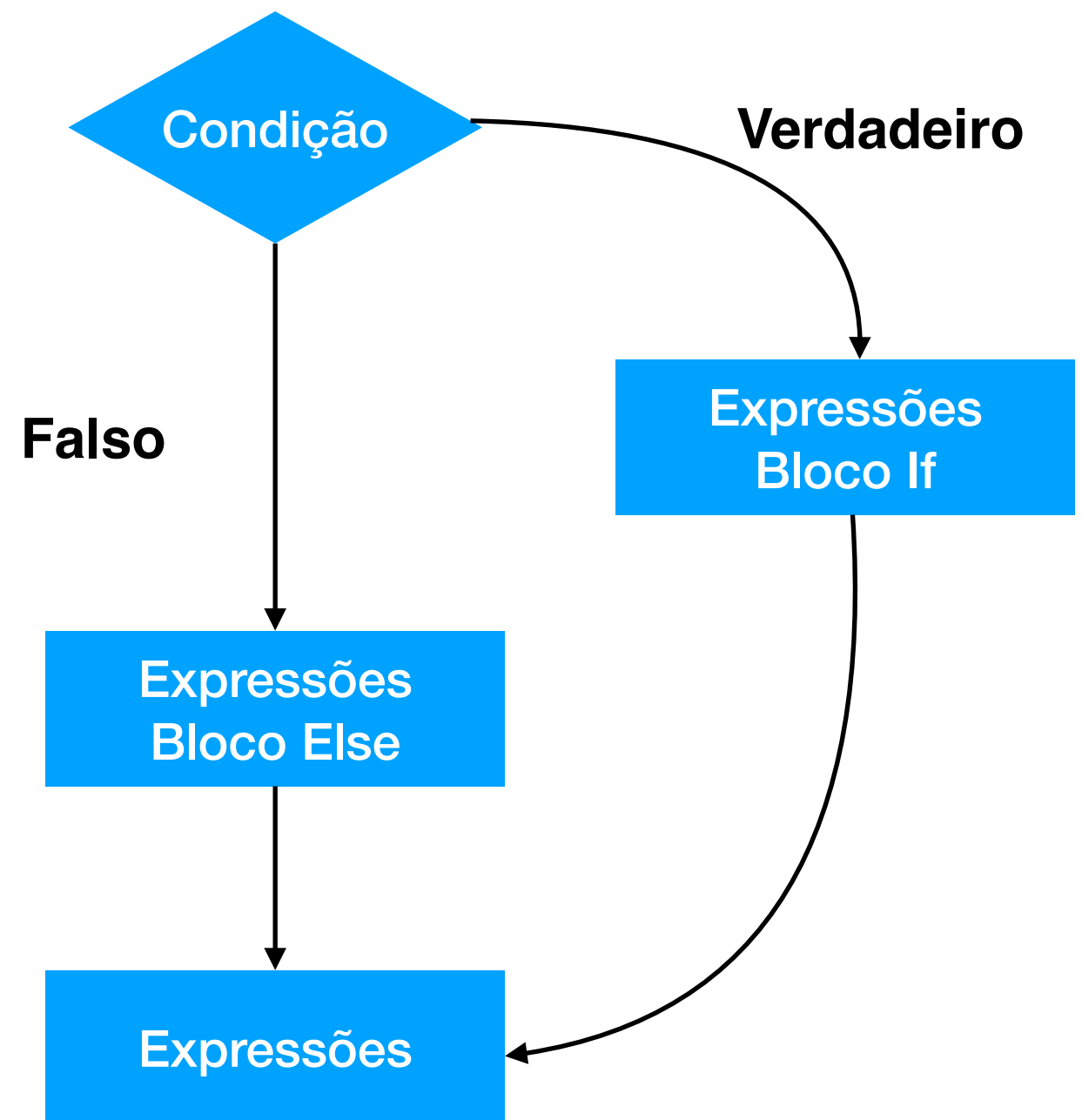
Operadores Lógicos

- Também fazem comparações
- Operandos de operadores lógicos são avaliados como lógicos (0 ou 1), e não como quantidades numéricas

Operação	Operador em Python	Operador em C
Lógico E	and	&&
Lógico OU	or	
Lógico NÃO	not	!

Se-Senão (*If-Else*): Desvio Condicional Composto

```
int main() {  
    if (<condição>) {  
        ...  
    }  
    else {  
        ...  
    }  
    return 0;  
}
```



Se-Senão (*If-Else*): Desvio Condicional Composto (Cont.)

```
int main() {  
  
    if (<condição1>) {  
        ...  
    }  
    else if (<condição2>) {  
        ...  
    }  
    else if (<condição3>) {  
        ...  
    }  
    else {  
        ...  
    }  
  
    return 0;  
}
```

O bloco **else if** é opcional, podendo não existir ou existir vários.

Em um desvio condicional composto, apenas 1 bloco será executado.

Exemplo 4: Pode Usar o Brinquedo?

```
1. #include <stdio.h>
2.
3. int main () {
4.     int altura_centimetros, idade;
5.
6.     printf("Entre com a sua idade e altura (em centimetros): \n");
7.     scanf("%d %d", &idade, &altura_centimetros);
8.
9.     if (idade < 12) {
10.        printf("Voce nao tem idade para usar o brinquedo!\n");
11.    }
12.    else {
13.        if (altura_centimetros < 150) {
14.            printf("Voce nao tem altura para usar o brinquedo!\n");
15.        }
16.        else {
17.            printf("Voce pode usar o brinquedo!\n");
18.        }
19.    }
20.    return 0;
21.}
22.
```

Exemplo 5: Comando Switch

```
1. #include <stdio.h>
2.
3. int main () {
4.     int op;
5.
6.     printf("Entre com uma opcao: \n")
7.     scanf("%d", &op);
8.
9.     switch(op) {
10.        case 1:
11.            printf("Opcao 1 selecionada!\n");
12.            break;
13.
14.        case 2:
15.            printf("Opcao 2 selecionada!\n");
16.            break;
17.
18.        default:
19.            printf("Opcao invalida! Entre com um valor valido!\n");
20.            break;
21.    }
22.
23.    return 0;
24. }
```

```
Entre com uma opcao:
1
Opcao 1 selecionada!

Entre com uma opcao:
99
Opcao invalida! Entre com um valor valido!
```

Operador Ternário

Programadores usam o operador ternário em C para tomar decisões *inplace* de expressões condicionais **if-else**

```
1. #include <stdio.h>
2.
3. int main () {
4.     int a = 10, b = 20, c;
5.
6.     c = (a < b) ? a : b;
7.
8.     printf("%d", c);
9.
10.    return 0;
11.}
12.
```

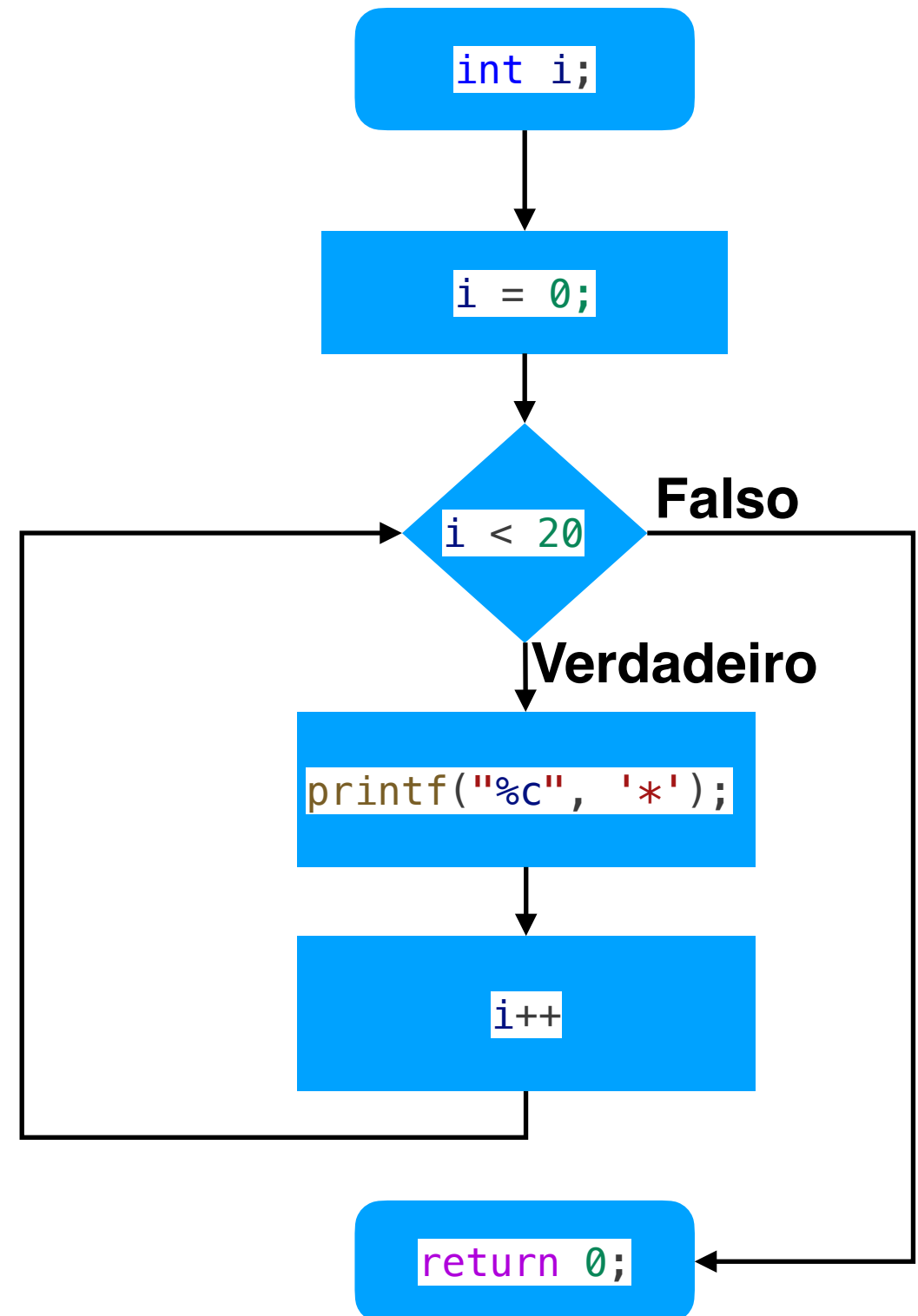
```
% ./a.out
10
```

Estruturas de Repetição

Laço Para (*For Loop*)

```
1. #include <stdio.h>
2.
3. int main () {
4.     int i;
5.
6.     for(i = 0; i < 20; i++)
7.         printf("%c", '*');
8.
9.     printf("\n");
10.
11.     return 0;
12. }
```

% ./a.out



Sintaxe Inicialização: Exemplos

`for(i = 0;;) {`  **Inicialização da variável i com o valor 0.**

Inicialização da variável i com o valor 0 e da variável j com 1.

`for(i = 0, j = 1;;) {`

`for(i = j;;) {`  **Inicialização da variável i com o valor de j.**

Nenhuma inicialização é feita.

`for(;;) {`

Sintaxe Condição: Exemplos

`for( ; i < 20; ) {`
`}`  **Condição verifica se a variável i é menor que 20.**

Expressão lógica verifica se a variável i é menor que 20 e também a variável j é maior que 10.

 `for( ; i < 20 && j > 10; ) {`
`}`

`for( ; i < j; ) {`
`}` 

Expressão lógica verifica se a variável i é menor que j.

Nenhuma expressão lógica. O laço será executado indefinidamente.

 `for(;;) {`
`}`

Sintaxe Incremento: Exemplos

```
for(;; i += 1) {
```

**Incremento de 1 à variável i
ao final de cada execução
do laço.**

**Incremento de 1 à variável i
e -2 à variável j ao final de
cada execução do laço.**

```
for(;; i += 1, j -= 2) {
```

```
for(;; i++) {
```

**Incremento de 1 à variável i
ao final de cada execução
do laço.**

**Nenhum incremento
realizado ao final de cada
execução do laço.**

```
for(;;) {
```

Exemplo 6: Pares de 600 a 1

```
1. #include <stdio.h>
2.
3. int main () {
4.     int i;
5.
6.     for(i = 600; i > 1; i = i - 2)
7.         printf("%d ", i);
8.
9.     printf("\n");
10.
11.     return 0;
12. }
```

600	598	596	594	592	590	588	586
584	582	580	578	576	574	572	570
568	566	564	562	560	558	556	554
552	550	548	546	544	542	540	538
536	534	532	530	528	526	524	522
520	518	516	514	512	510	508	506

...

Operador Incremento/Decremento

Pré-fixado:

```
n = 5;  
x = ++n;  
printf("n=%d; x=%d", n, x);
```



```
n = 5;  
n = n + 1;  
x = n;  
printf("n=%d; x=%d", n, x);
```

Pós-fixado:

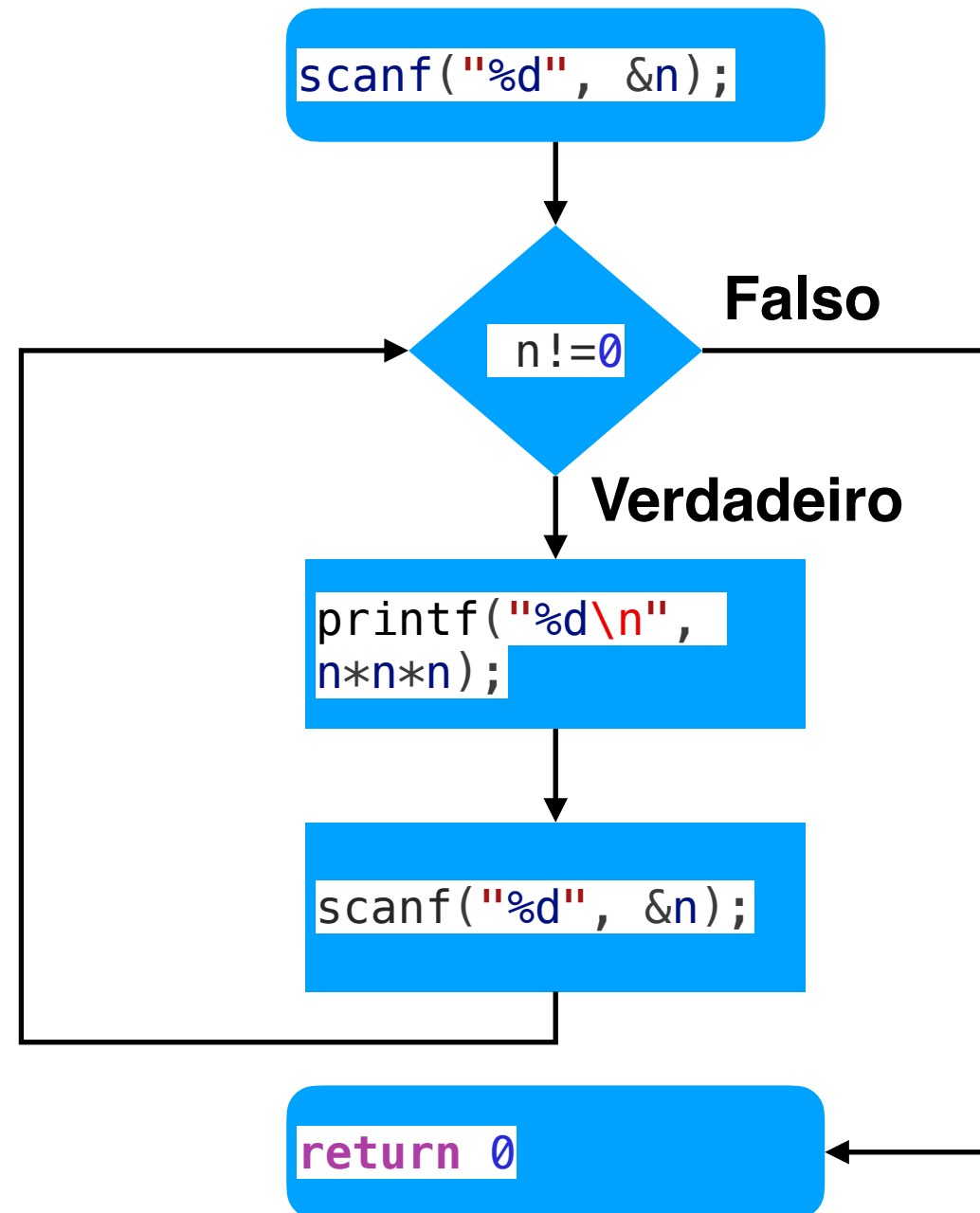
```
n = 5;  
x = n++;  
printf("n=%d; x=%d", n, x);
```



```
n = 5;  
x = n;  
n = n + 1;  
printf("n=%d; x=%d", n, x);
```

Laço Enquanto (*While Loop*)

```
1. #include <stdio.h>
2.
3. int main () {
4.     int n;
5.
6.     scanf("%d", &n);
7.
8.     while (n != 0) {
9.         printf("%d\n", n*n*n);
10.        scanf("%d", &n);
11.    }
12.
13.    return 0;
14.}
```



Referências

- DEITEL, Paul; DEITEL, Harvey; C++ How to Program (9 edition). Pearson, 2016.